SLA-JEC: Structured Localization and Auxiliary Adaptation for Japanese Grammatical Error Correction

Abstract

Category-conditioned Japanese grammatical error correction identifies an erroneous span and supplies its replacement while preserving the surrounding text. However, limited learner annotations make it difficult to learn both precise error boundaries and varied grammatical revisions. We therefore propose SLA-JEC, a system for structured localization and auxiliary adaptation that provides focused correction suggestions for Japanese learning tasks. We extend a character gated recurrent unit with sparse local predictors, a context-dependent localization gate and constrained edit search. We then adapt a pretrained generator with simulated errors and use a likelihood rule to select between its replacement and the compact editor's output. We construct CAMC-JL from 759 learner-writing examples and release its metadata and numerical results alongside method code and trained weights. On the CAMC-JL test set, the system achieves 33.04% exact span-and-replacement agreement, compared with 27.97% for the matched source-only generator and 14.31% for the standalone recurrent model. These results show that structured localization and auxiliary adaptation complement each other in improving local Japanese correction.

Keywords: Japanese grammatical error correction, structured localization, character GRU, auxiliary adaptation, learner-writing dataset

1 Introduction

Japanese grammatical error correction identifies grammatical errors in learner writing and supplies appropriate revisions. We focus on correction with an error category as input. Given a text and a category such as tense or honorific usage, the system returns one source interval and its replacement. It preserves the surrounding text and presents the change as a local suggestion. This output format supports learning activities that ask a student to inspect a particular grammatical construction.

The task requires both precise localization and an appropriate replacement. Japanese grammatical distinctions involve short character sequences within longer contexts, while different lexical constructions require different revisions. With limited learner annotations, a model must learn where an error begins and ends as well as how to modify it. A compact character model learns contextual representations, but its observed edit inventory limits the replacements it can produce. A pretrained generator offers a larger language model, while a fixed interval gives its correction a clear target.

We propose SLA-JEC to connect these two capabilities. We use a character gated recurrent unit (GRU) as the compact model and combine its predictions with sparse local features. A context gate mixes localization probabilities from original and cropped training contexts. The resulting localizer selects one interval before either editor produces a replacement. We then compare a constrained compact edit with a replacement from an adapted generator. The generator supplies the final revision only when it produces a valid output with a higher conditional likelihood.

We construct CAMC-JL as a 759-example dataset for this task. It brings together learner-writing annotations from Natane and ICJS/TUFS under a common set of error categories and correction rules. We also release the CAMC-JL metadata and numerical-results dataset, including 10,062 per-example evaluation records, together with method code and trained weights. This resource connects dataset composition, model comparisons and numerical reanalysis in one project.

Our technical contributions concern the localization model and the revision procedure. Sparse–recurrent fusion and context-dependent gating extend the GRU's localization decisions, while constrained search produces a changed replacement within the selected interval. Auxiliary adaptation expands the generator's correction experience, and likelihood selection retains the compact editor as an alternative. We evaluate these components through retrieval and supervised baselines, component ablations and a matched comparison that holds the generator architecture, localization model and target-fitting schedule fixed. SLA-JEC achieves 33.04% ExactMatch, a 5.07-percentage-point increase over the matched source-only generator. The following experiments examine where that improvement occurs and how computation and context affect the system.

2 Related work

2.1 Correction models and local editing

Bryant et al. review generation, editing and evaluation methods for grammatical error correction [1]. Omelianchuk et al. introduce GECToR, which uses a Transformer encoder and token transformations to express corrections as edits [2]. Wu et al. predict position-offset labels to combine lexical generation with copying [3]. These approaches motivate an explicit edit representation that preserves useful source text.

Homma and Komachi examine autoregressive and non-autoregressive Japanese correction, including accuracy on incomplete sentences and inference speed [4]. Arai et al. retrieve examples that reflect a Japanese learner's grammatical error type [5]. We combine category information with local editing so that each output identifies both the target fragment and its replacement. Our experiments use a common one-span interface for retrieval, compact editing and generation.

2.2 Learner data and model adaptation

Koyama et al. construct a Japanese learner correction corpus and evaluate statistical and neural translation models [6]. Suzuki et al. construct a quality-estimation dataset for Japanese correction [7]. These resources address complementary aspects of correction data and evaluation. CAMC-JL organizes source annotations around a category-conditioned local edit and reports localization and revision separately.

Kimn et al. release rule-based synthetic Japanese error data [8]. Omelianchuk et al. also use synthetic pretraining followed by task-data fine-tuning [2]. Hu et al. introduce low-rank adaptation (LoRA) for efficient adaptation of pretrained models [9]. We use LoRA to adapt a replacement generator while retaining a compact localizer. These methods provide edit representations and efficient adaptation procedures. The remaining design problem is how to combine precise interval selection with flexible replacement generation in one category-conditioned correction procedure under limited learner supervision.

3 SLA-JEC

3.1 Task definition and overview

Let $x = (x_0, \ldots, x_{n-1})$ denote the input characters and c the error category. We predict a nonempty interval $[s, e)$ and a replacement y. We limit $e - s$ to 40 characters and require y to differ from $x_{s:e}$. We also require y to contain at least one character and at most $\max(44, 4(e-s))$ characters. We preserve the surrounding text by forming

$$x' = x_{0:s} \parallel y \parallel x_{e:n}, \tag{1}$$

where $\parallel$ denotes concatenation.

Figure 1 presents the system. We train the compact localization and editing components on CAMC-JL source supervision. We train the generator first on auxiliary pairs and then on the same source supervision. During inference, the compact frontend predicts the interval and a candidate replacement. The generator receives that interval and proposes a second replacement. A likelihood comparison chooses the output.

3.2 GRU fusion and context-dependent localization

We combine learned character representations with explicit local features to locate short grammatical errors. The bidirectional GRU uses 48-dimensional character embeddings, 16-dimensional category embeddings and 48 hidden units in each direction [10]. Its shared representation feeds a BIO classifier, a character-edit head and a prefix-emission head. The BIO labels identify the beginning of the target span, its interior and characters outside it.

The sparse localizer uses characters within a radius of three, local bigrams and trigrams, script identity, category interactions and coarse relative position. These features supply explicit local patterns alongside the recurrent representation. Let $p_{G,i}$ and $p_{L,i}$ denote the aligned GRU and sparse BIO distributions at position i. The original-context fusion uses

$$q_i^{\mathrm{orig}}(k) = \tfrac{1}{2}\{p_{G,i}(k) + p_{L,i}(k)\}, \qquad k \in \{O, B, I\}. \tag{2}$$

To account for changes in the amount of surrounding text, we train another sparse localizer on cropped contexts and remove its coarse position features. This model produces $p_{C,i}$. We combine it with the same GRU to obtain $q_i^{\mathrm{crop}} = (p_{G,i} + p_{C,i})/2$. A logistic gate predicts a mixture weight $g(x)$ from six input features. They comprise log context length, its square, whitespace fraction, final punctuation and leading and trailing whitespace indicators. We

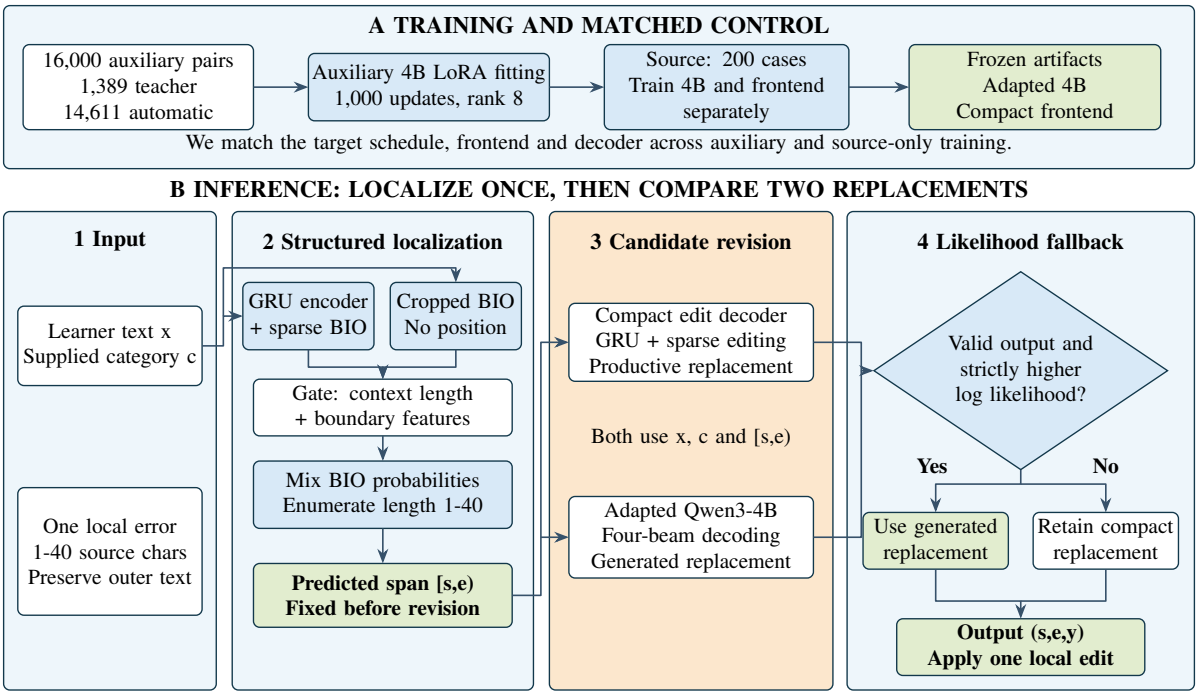


Fig. 1 SLA-JEC training and inference. We train the compact frontend and generator separately. At inference, the frontend combines recurrent and sparse BIO predictions, applies a context gate and fixes one interval. Both editors then propose a replacement for that interval. The likelihood rule chooses a valid generated replacement when its score exceeds the compact replacement's score. Otherwise, it retains the compact output. The matched source-only control uses the same frontend and target-fitting schedule and starts generator adaptation directly from source data.

standardize these features using fitting-data statistics and train the gate to identify shortened contexts. The final distribution is

$$q_i(k) = (1 - g(x))q_i^{\text{orig}}(k) + g(x)q_i^{\text{crop}}(k). \tag{3}$$

This input-dependent mixture follows the general use of gates to combine local experts [11].

We select the interval that maximizes its log-probability advantage over the all-outside label sequence,

$$S(s,e) = \log \frac{q_s(B)}{q_s(O)} + \sum_{i=s+1}^{e-1} \log \frac{q_i(I)}{q_i(O)}. \tag{4}$$

Enumeration over lengths 1–40 takes $O(40n)$ time. For equal scores, we prefer the shorter interval and then the earlier interval. We hold this interval fixed throughout replacement generation and selection.

3.3 Compact replacement generation

We use character edit actions to generate the compact candidate. Minimum-edit alignment converts each fitting pair into a prefix emission and one action per source character. An action keeps the character, deletes it or emits a learned string. The sparse editor adds interval length, within-span position and boundary features to its local character features.

We align the GRU and sparse edit distributions over the union of their fitting-data labels and average their probabilities. Best-first search examines up to 256 edit states and returns the highest-ranked encountered replacement that satisfies the task constraints. We call such a changed, nonempty and length-bounded replacement a productive edit. When search finds no productive edit, the compact component returns an empty correction. This search rule gives the model an explicit alternative to copying the original span unchanged.

The original GRU stores 97,833 parameters. Adding sparse localization and editing gives the fast fusion 3,565,554 parameters and coefficients. The crop localizer and gate bring the complete frontend to 4,234,774 stored values. The improvement therefore changes the GRU-based correction system through complementary predictors and decoding.

3.4 Auxiliary adaptation and likelihood selection

The compact editor learns its replacement inventory from source examples. We add a pretrained generator to produce revisions beyond this inventory. We use Qwen3-4B-Instruct-2507 [12] and fit rank-8 LoRA adapters [9]. Auxiliary fitting uses simulated errors, followed by source fitting on CAMC-JL. For the matched control, we start from the same untrained adapter and perform source fitting with the same schedule.

We provide the generator with the error category and input text, placing `<error>` and `</error>` around the predicted interval. The instruction requests only replacement text. Four-beam deterministic decoding produces a candidate y_G. We strip outer whitespace and accept the candidate as valid when it reaches the end token, satisfies the productive-edit constraints and contains no internal line break.

We compare y_G with the compact candidate y_C under the same adapted generator. For response tokens $t_1, \ldots, t_m$, including the end token, we compute

$$\ell_\theta(y \mid x, c, s, e) = m^{-0.6} \sum_{j=1}^{m} \log p_\theta(t_j \mid t_{<j}, x, c, s, e). \tag{5}$$

The final rule is

$$y^* = \begin{cases} y_G, & \text{if } y_G \text{ is valid and } \ell_\theta(y_G) > \ell_\theta(y_C), \\ y_C, & \text{otherwise.} \end{cases} \tag{6}$$

We retain the compact candidate on ties. An empty final correction leaves the input text unchanged. This rule lets auxiliary adaptation affect replacement choice while both alternatives share the same localization decision.

4 Experimental setup

4.1 CAMC-JL construction

We construct CAMC-JL from 759 learner-writing examples in local Natane and ICJS/TUFS snapshots. Table 1 summarizes its composition. We map source labels to ten categories covering aspect, tense, style, passive, transitivity, honorific, benefactive, demonstrative, sentence connection and causative.

Our construction starts from 1,823 raw records. Filtering unusable spans and replacements retains 1,376 records, context filtering retains 901, and category normalization yields 759. We remove unchanged replacements, source-editorial markup and edits that exceed the length limits in Section 3.1. We retain input windows of at least 55 characters. For ICJS/TUFS, we include up to 90 characters on each side of the target, trim outer whitespace and adjust the offsets. We require an unambiguous target occurrence and exclude ICJS/TUFS windows with multiple targets of the same category.

4.2 Development and comparison protocol

We organize the 200 development examples into 121 connected components using shared source documents, available learner identities and normalized windows. Three grouped folds contain 67, 67 and 66 validation examples. For each fold, we rebuild vocabularies, edit inventories and sparse features from its fitting portion. Final refits use all 200 development examples and seeds 20260927, 20260928 and 20260929.

Within the SLA-JEC development pipeline, we rank component candidates by their minimum ratio to the current model across the six evaluation metrics. We break ties by mean ratio, ExactMatch, SoftMatch and a fixed configuration order. For context and generator selection, we additionally require non-regression on original and pooled crop means. We construct fitting crops with widths 32 and 64 and left-context fractions $0, 1/3, 2/3, 1$. Validation crops use fractions $1/4, 3/4$, giving 800 views of the same 200 development parents.

We retain the original 559-case test partition for comparisons with the existing models. Earlier development used this benchmark, and 313 test cases share a source document with development. We therefore report historical

Table 1: CAMC-JL composition. We use the development portion for fitting and model selection and the historical test portion for the reported comparisons.

Source	Development	Historical test	Total
Natane	94	270	364
ICJS/TUFS	106	289	395
Total	200	559	759

comparisons and analyze source and correction-pair subsets. The test contains 60 cases with a typed span–replacement pair seen in development and 499 cases with an unseen pair. A further subset contains 207 cases without a shared available source unit. We define source units by ICJS/TUFS learner identity and Natane document identity, giving 194 test clusters.

4.3 Auxiliary training data

Kimn et al. provide simulated Japanese error pairs [8]. We retain 6,297 valid teacher-authored pairs and group connected sentence families before assigning 5,093 pairs to training, 636 to validation and 568 to a reserved partition. We combine the training portion with 54,674 eligible automatically generated pairs. Normalized-text overlap checks exclude the protected CAMC-JL and reserved texts from this auxiliary pool.

We sample 16,000 distinct pairs from the 59,767 eligible training pairs using a fixed random seed of 20260928. The sample contains 1,389 teacher-authored and 14,611 automatic pairs. We fit the auxiliary adapter on this sample and start each source-stage seed from that same adapter. This sequence gives the generator additional correction patterns before it learns the CAMC-JL task.

4.4 Training and decoding settings

We train the GRU with whole-span negative log likelihood, weighted BIO cross-entropy, weighted edit loss and prefix-emission loss. The BIO term has weight 0.2. We use inverse-frequency BIO weights and clipped square-root inverse-frequency edit weights in $[0.25, 4]$. GRU training uses AdamW, batch size 16, dropout 0.1, gradient clipping at 1 and 80 epochs. The learning rate reaches 0.003 after five warm-up epochs and follows cosine decay to one tenth of that value. Weight decay is 10^{-4} for matrix weights and zero for embeddings and vectors.

We L2-normalize the sparse features and fit logistic-loss stochastic gradient descent with L2 regularization 10^{-4}, at most 2,000 iterations and tolerance 10^{-4}. BIO fitting balances the classes. The context gate uses balanced L2 logistic regression with $C = 1$, tolerance 10^{-8} and at most 1,000 iterations. We fit the gate's standardization statistics within each training fold.

Table 2 gives the generator settings. We use the Qwen checkpoint revision `cdbee75f17c01a7cc42f958dc650907174af0554`. Its base has 4,022,468,096 parameters, and the LoRA adapters add 16,515,072 trainable parameters. We use a BF16 base and float32 adapters. Including the frontend, the full system stores 4,043,217,942 model values.

4.5 Metrics, baselines and uncertainty

We report six example-mean metrics. SpanMatch (SM) measures exact interval agreement, and SpanOverlap (SO) measures intersection over union. RevisionMatch (RM) measures exact replacement agreement after NFKC normalization and whitespace collapse. RevisionSimilarity (RS) equals one minus character Levenshtein distance divided by the larger normalized replacement length. SoftMatch multiplies SO by RS, and ExactMatch multiplies SM by RM. We express means as percentages.

For consistency with the existing benchmark, the correction scorer locates the emitted source substring in the input. For repeated occurrences, it chooses the occurrence with greatest reference overlap and then the smallest

Table 2: Generator training and inference settings. Both matched 4B arms use the same source stage and decoder.

Setting	Value
LoRA	Rank 8, scale 16, dropout 0.05
Target projections	Attention q/k/v/o and feed-forward gate/up/down
Auxiliary stage	1,000 updates, peak learning rate 2×10^{-4}
Source stage	10 epochs, peak learning rate 10^{-4}, fresh AdamW optimizer
Source schedule	130 updates for 200 examples, 390-update learning-rate horizon
Batch and optimizer	Microbatch 2, effective batch 16, weight decay 0.01, clipping 1
Learning-rate schedule	5% warm-up, cosine decay to multiplier 0.1
Training objective	Response-token cross-entropy including the end token
Input length	At most 512 tokens for the encoded training example
Decoding	Four beams, length penalty 0.6, early stopping, no sampling
Generation limit	$\min(128, \max(16, 4(e-s)+8))$ new tokens
Inference batches	Four requests for generation and likelihood scoring

distance from the reference start. We also score the saved locator offsets directly. These offset scores distinguish the localizer's decision from the correction record, which is empty when the editor returns no valid replacement.

Our baselines cover correction memory, TF–IDF and BM25 retrieval, sparse localization with nearest-revision retrieval [13], a linear edit tagger and the standalone character GRU. We also compare compact fusion with adapted 0.6B and 1.7B generators. The main auxiliary-data comparison uses two 4B systems with identical frontends, target schedules and decoding rules. We average neural results over the three refit seeds.

For paired intervals, we average seed-matched differences within each test example and draw 20,000 bootstrap samples of the 194 source-unit clusters with random seed 20260928. Each sample retains all examples in the drawn clusters. We use percentile endpoints at $0.05/(2\times6)$ and $1-0.05/(2\times6)$ to adjust six metrics within each comparison family. These intervals quantify paired variation conditional on the fitted models and evaluation set. They cover that comparison family rather than the earlier adaptive development sequence.

5 Results

5.1 Comparison with retrieval and supervised baselines

SLA-JEC achieves 33.04% ExactMatch on CAMC-JL (Table 3). Retrieval baselines reach approximately 10%, while the standalone GRU reaches 14.31%. Fast GRU–linear fusion increases this score to 17.05%. Adapted generators further improve exact correction, with the auxiliary 1.7B system reaching 26.77% and the matched source-only 4B system reaching 27.97%.

Table 3: CAMC-JL test results (%). Neural methods average three seeds. Retrieval and linear methods use deterministic fitting. Higher values indicate greater annotation agreement.

Method	SM	SO	RM	RS	Soft	Exact
Frequency memory	10.73	12.59	8.77	15.88	9.54	8.23
TF–IDF retrieval	12.88	14.39	10.38	16.98	11.54	10.02
BM25 retrieval	13.06	14.52	10.55	16.92	11.57	10.20
Linear + nearest revision	66.91	74.28	12.16	33.27	28.86	8.41
Linear edit tagger	66.91	74.28	9.84	44.13	41.79	9.84
Character GRU	66.79	67.85	14.31	40.84	38.10	14.31
Fast GRU–linear fusion	75.97	77.50	17.05	46.71	45.29	17.05
Auxiliary-16k, 0.6B	75.97	77.50	21.65	44.72	43.28	21.65
Coverage-37,888, 0.6B	75.97	77.50	22.12	45.80	44.34	22.12
Auxiliary-16k, 1.7B	75.97	77.50	26.77	48.89	47.53	26.77
Context gate, 1.7B	76.45	77.90	26.71	49.00	47.71	26.71
Context gate, source-only 4B	76.45	77.90	28.15	51.06	49.03	27.97
SLA-JEC, auxiliary 4B	**76.45**	**77.90**	**33.09**	**53.26**	**51.77**	**33.04**

The six metrics show why localization and replacement need separate analysis. The linear localizer reaches 66.91% SpanMatch, but a nearest-revision editor yields only 8.41% ExactMatch. Replacing retrieval with edit tagging raises revision similarity while exact correction remains below the GRU. Sparse features therefore provide useful location information, but a successful correction also requires an editor that chooses the right replacement.

5.2 Contribution of compact components

Table 4 isolates localization fusion, revision fusion and productive search. Localization fusion raises SpanMatch from 66.79% to 75.97% and ExactMatch from 14.31% to 15.80%. Revision fusion alone retains the GRU's locations and raises ExactMatch to 15.44%. Combining the predictors with productive search reaches 17.05%, a 2.74-point gain over the GRU with a six-metric-adjusted paired interval of [1.26,4.44].

Productive search increases ExactMatch from 16.16% for fused argmax decoding to 17.05%. SoftMatch falls from 46.54% to 45.29%. The search rule thus favors a valid changed correction and improves exact agreement at the cost of some partial string agreement. The context gate subsequently raises full-test SpanMatch from 75.97% to 76.45%. We examine its behavior under a different context distribution in Section 5.6.

Figure 2 locates the remaining error burden. SLA-JEC achieves the correct location but an incorrect replacement on 43.41% of cases and an incorrect location on 23.55%. The matched source-only system has the same localization error share and a larger 48.48% correct-location, incorrect-replacement share. The ratio of ExactMatch to SpanMatch consequently rises from 36.58% to 43.21%. Auxiliary adaptation improves the replacement decision after localization, while replacement errors remain the larger group.

Table 4: Compact component ablations on the 559 test cases (%). Localization fusion averages BIO predictions. Revision fusion averages edit predictions. Productive search applies the changed-output constraints.

Method	SM	SO	RM	RS	Soft	Exact
GRU	66.79	67.85	14.31	40.84	38.10	14.31
Linear, productive	66.91	74.28	11.63	43.21	41.19	11.63
Localization fusion	75.97	77.50	15.80	44.52	43.11	15.80
Revision fusion	66.79	67.85	15.44	42.90	40.10	15.44
Fusion, argmax	75.97	77.50	16.16	48.16	46.54	16.16
Fusion, productive	75.97	77.50	17.05	46.71	45.29	17.05

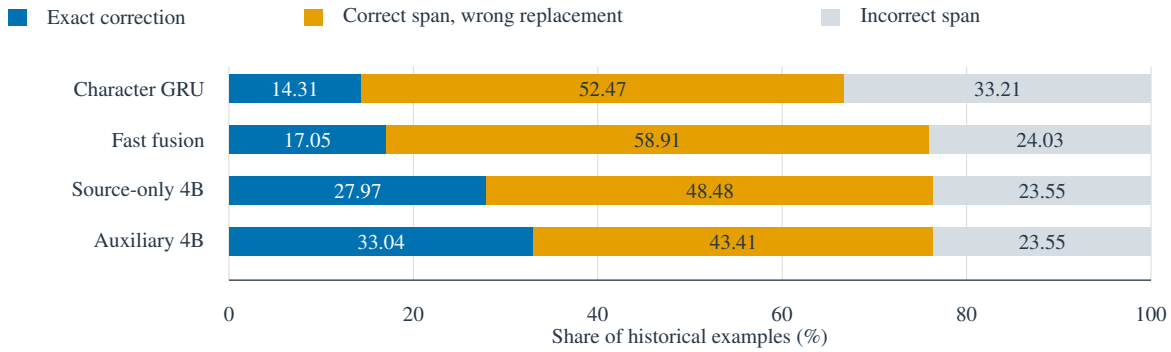


Fig. 2 Decomposition of correction outcomes. We compute exact correction as ExactMatch, correct location with an incorrect replacement as SM minus ExactMatch, and incorrect location as one minus SM. Each bar partitions the three-seed aggregate results into these three outcomes.

5.3 Matched auxiliary adaptation

Auxiliary adaptation improves ExactMatch by 5.07 percentage points over the matched source-only 4B system (Table 5 and Fig. 3). The six-metric-adjusted 95% interval is [1.18,9.32]. RevisionMatch also has a positive interval. RevisionSimilarity and SoftMatch increase in mean, while their adjusted intervals include zero. The two systems share their localization predictions, so both location differences equal zero.

Table 5: Auxiliary-minus-source-only differences for the matched 4B systems. We adjust the paired cluster-bootstrap intervals across six metrics.

Metric	Difference (percentage points)	Simultaneous 95% interval
SpanMatch	0.000	[0.000, 0.000]
SpanOverlap	0.000	[0.000, 0.000]
RevisionMatch	4.949	[1.068, 9.240]
RevisionSimilarity	2.193	[-0.795, 5.397]
SoftMatch	2.738	[-0.054, 5.823]
ExactMatch	5.069	[1.179, 9.324]

Across the three seeds, the auxiliary model's ExactMatch ranges from 32.20% to 33.63%, compared with 27.55% to 28.26% for source-only adaptation. Each matched seed improves all four revision means. The common frontend and fitting schedule make this comparison directly informative about the auxiliary training stage.

5.4 Correction familiarity and source subsets

The auxiliary gain occurs mainly on correction pairs that do not appear in development (Fig. 4). For 499 unseen-pair cases, ExactMatch increases from 20.57% to 26.45%, with an adjusted difference interval of [1.70,10.52] points. For the 60 seen-pair cases, it decreases from 89.44% to 87.78%, with interval [-8.20,6.11]. Weighting the two groups by their sizes assigns approximately +5.25 points of the full-set change to unseen pairs and -0.18 points to seen pairs.

For the 207 examples without a shared available source unit, ExactMatch increases from 25.76% to 35.43%. The corresponding difference is 9.66 points with interval [2.10,18.41]. Together, these subsets show that the aggregate

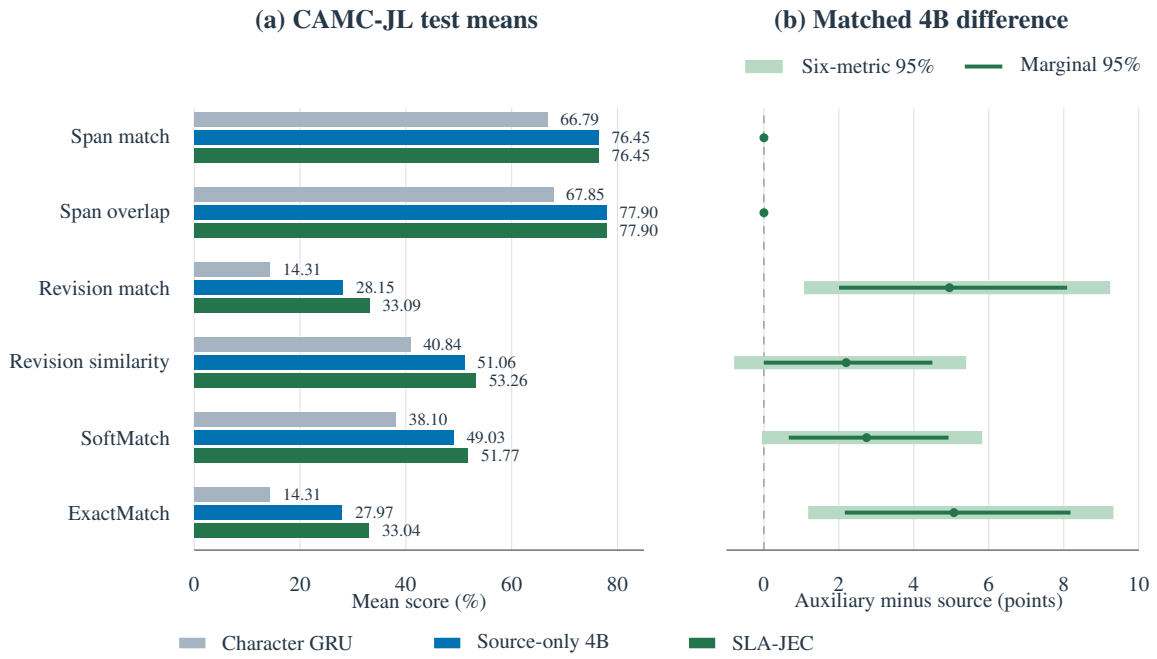


Fig. 3 Matched 4B comparison. The left panel shows three-seed metric means. The right panel shows auxiliary-minus-source-only differences with marginal and six-metric-adjusted 95% intervals from 20,000 paired source-cluster resamples.

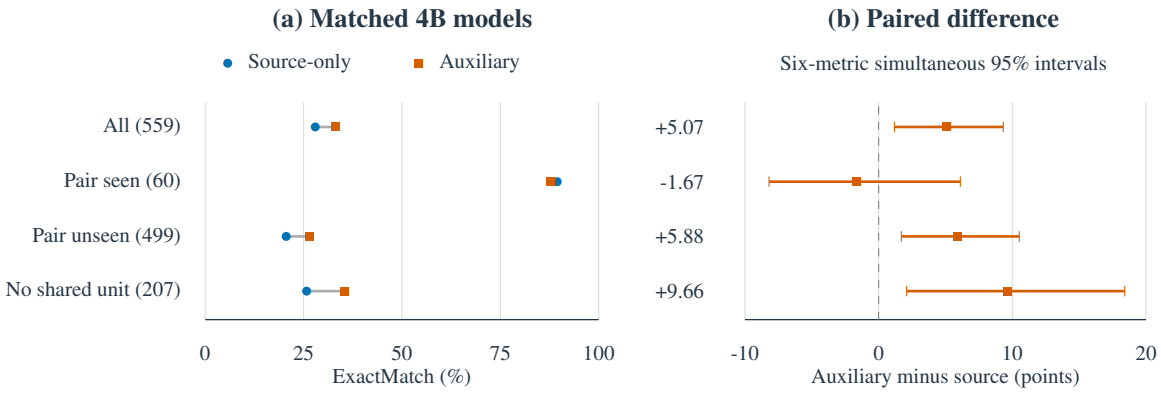


Fig. 4 ExactMatch by source and correction-pair subset. Panel (a) shows matched 4B means, and panel (b) shows differences with six-metric-adjusted intervals within each subset. Seen and unseen pairs partition the full set. The no-shared-unit subset overlaps them. Source-unit cluster counts are 194, 44, 190 and 95 in the displayed order.

improvement extends beyond correction pairs from development. The wider source-subset interval reflects its smaller number of examples and source clusters.

5.5 Target fitting and replacement selection

Development controls show that auxiliary training and source adaptation serve complementary roles (Table 6). Auxiliary training alone with fallback reaches 18.00% ExactMatch on original contexts. Adding source fitting raises this score to 27.50%. For the latter model, retaining the compact candidate raises ExactMatch from the generator-only 25.00% to 27.50% and SoftMatch from 40.45% to 47.17%.

Figure 5 shows the same benefit under shortened contexts. Fallback raises auxiliary-plus-source ExactMatch from 21.25% to 24.88% on pooled crop views. Relative to the source-only control, the selected system improves ExactMatch by 2.00 points on original contexts and 3.00 points on pooled crops. An individual left-positioned 64-character crop has a 0.0766-point RevisionSimilarity decline, so the pooled gain includes a small local tradeoff.

Table 6: Development controls (%). Orig. uses 200 out-of-fold contexts. Crop uses 800 transformed views of those same examples. SM/SO score emitted correction records, so invalid replacements reduce these scores even when locator offsets remain fixed.

Variant	View	SM	SO	RM	RS	Soft	Exact
Source-only + fallback	Orig.	70.50	72.47	25.50	48.28	46.45	25.50
Source-only + fallback	Crop	69.38	71.52	22.13	45.73	43.16	21.88
Auxiliary only, generator	Orig.	47.00	48.43	9.00	25.64	24.61	9.00
Auxiliary only, generator	Crop	41.25	42.66	8.50	22.51	21.64	8.50
Auxiliary only + fallback	Orig.	70.50	72.47	18.00	42.04	40.85	18.00
Auxiliary only + fallback	Crop	69.38	71.52	18.75	41.90	40.45	18.75
Auxiliary + source, generator	Orig.	58.50	59.51	25.00	41.74	40.45	25.00
Auxiliary + source, generator	Crop	54.87	56.11	21.63	37.65	36.07	21.25
Auxiliary + source + fallback	Orig.	70.50	72.47	27.50	48.73	47.17	27.50
Auxiliary + source + fallback	Crop	69.38	71.52	25.25	46.59	44.53	24.88

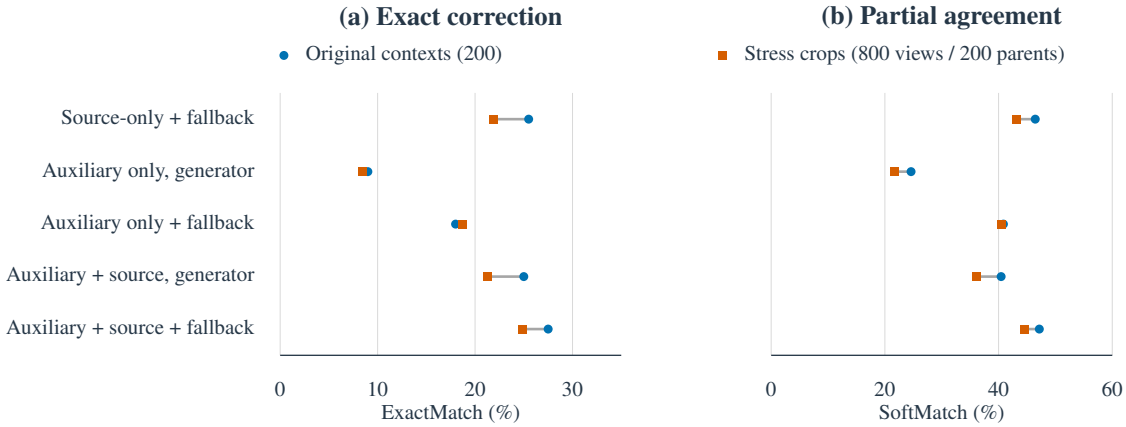


Fig. 5 Original and cropped-context development results. Each segment connects the two evaluation conditions for one variant. All variants share the saved localization offsets. The cropped condition contains four views of each original development example.

5.6 External localization

We construct an external localization evaluation from Teramura's collection of Japanese learner errors [14]. The projection contains 803 records from 221 learners across nine categories. Its abstract correction labels support interval evaluation, and we score predicted offsets directly. We use this corpus as a diagnostic of localization under different text conditions. Context-gate development followed the initial inspection of this resource.

Table 7: External localization on 803 Teramura examples from 221 learners. Values show three-seed means (%). We score actual predicted offsets.

Locator	Offset exact match	Offset overlap
Character GRU	1.04	7.59
Sparse linear	0.25	3.07
Fast fusion	0.12	2.08
Context-gated frontend	0.33	6.85

All four localizers have low exact agreement on Teramura (Table 7). The context gate reaches 0.33% exact location agreement and 6.85% overlap, compared with 0.12% and 2.08% for fast fusion. The standalone GRU reaches 1.04% and 7.59%. Cropped training and gating therefore recover some overlap over fast fusion but leave a substantial transfer problem.

The input distributions help characterize this problem. Median text length is 127.5 characters in CAMC-JL development and 30 in Teramura. Reference midpoints fall in the middle fifth of the text for 92.00% of development

examples and 13.20% of external examples. The sparse and fused predictors concentrate their outputs near the development positions. These observations motivate broader natural-context coverage in subsequent localization training.

5.7 Computational cost

Fast fusion takes a mean 13.6 ms per example on one CPU thread, including feature extraction and excluding model loading. Its portable implementation averages 11.3 ms and reproduces all 1,677 three-seed test predictions. This component provides a small CPU correction option.

The full auxiliary 4B study uses one 16 GB RTX 4070 Ti SUPER. Auxiliary fitting takes 2,504.18 seconds, or about 41.7 minutes. Development-stage peak allocated and reserved memory reaches 9.788 and 15.777 GB. The historical batched GPU pass takes 525.23 seconds and reaches 9.682 and 15.743 GB.

Table 8: Model size and measured singleton latency. Timings use 12 fixed development inputs and seed 20260927, include the frontend and exclude loading. ExactMatch uses the 559-case test and three seeds. A dash indicates no measurement.

Pipeline	Values (10^9)	GPU (s)	CPU (s)	Exact (%)
0.6B, auxiliary 16k	0.605	0.270	10.141	21.65
0.6B, coverage 37,888	0.605	0.269	10.130	22.12
1.7B, auxiliary 16k	1.733	0.297	28.323	26.77
4B, source-only	4.043	0.538	—	27.97
4B, auxiliary 16k	4.043	—	—	33.04

Table 8 compares available singleton measurements across model sizes. The source-only 4B pipeline uses about 2.33 times as many stored values as the 1.7B pipeline and takes 0.538 seconds per input on GPU. Auxiliary adaptation improves 4B ExactMatch at the same stored model size. The auxiliary 4B row reports its measured accuracy, while the preceding paragraph reports its batched run cost. CPU timings for the smaller generators use float32 and one thread.

6 Discussion

The component results explain the division of work in SLA-JEC. Sparse–recurrent fusion improves the location estimate, and auxiliary adaptation improves the replacement at a fixed location. The compact candidate remains useful after generator adaptation, as the development fallback controls show. Together, these findings support an architecture that exposes localization and revision as separate decisions.

The results also identify the next modeling priority. Even with auxiliary adaptation, the system assigns 43.41% of examples to the correct-location, incorrect-replacement group. Exact agreement therefore depends strongly on replacement choice. The unseen-pair gains motivate broader correction coverage, while the seen-pair regressions motivate better selection between familiar compact edits and generated alternatives.

The evaluation conditions guide interpretation of these results. CAMC-JL uses one reference per case and measures agreement with that annotation. Historical benchmark reuse and shared source units limit estimates of transfer to new learners. Source-cluster intervals and subgroup comparisons describe variation within the available data. A future evaluation should collect learner-disjoint texts and compare multiple acceptable revisions. The Teramura results further motivate training and testing across natural context lengths.

For classroom-oriented development, the compact CPU model and the full GPU model offer different resource choices. The compact fusion supports fast local editing, while the auxiliary 4B system gives the highest exact correction among the local baselines in this study. A prospective study with learners and teachers would connect these computational results to the intended learning activities.

7 Conclusion

We propose SLA-JEC for Japanese grammatical error correction with an error category as input. The system combines GRU-based structured localization, sparse editing, a context gate and an auxiliary-adapted replacement generator. It selects between compact and generated revisions while preserving the chosen interval and surrounding text. On CAMC-JL, auxiliary adaptation increases ExactMatch from 27.97% to 33.04% under matched 4B conditions. Component analyses show complementary contributions from localization fusion and replacement

adaptation. We release the CAMC-JL metadata and numerical-results dataset with code and trained weights to support further comparison and development.

Statements and Declarations

The separate title page contains author information, funding and competing-interest declarations.

Data and code availability

We release the CAMC-JL metadata and numerical-results dataset, method code and trained weights through the project website at `https://panzerjagerwang.github.io/camc-jl-structured-correction/`. The versioned release is CAMC-JL v2.0.0. It contains 759 metadata records, opaque grouping labels, development folds and 10,062 evaluation records for six methods and three seeds. These files support numerical reanalysis of the reported model comparisons.

We license author-created metadata and results under CC BY 4.0, method code under MIT, and frontend weights and final adapters under Apache 2.0. Users obtain the pinned Qwen base separately. The learner texts, reference corrections and raw predictions remain with the source material under its access conditions. Source access is available through the Hinoki Project and the ICJS/TUFS corpus interface.

Ethics

We perform secondary computational analysis of existing learner-writing annotations. The study involves no new participant recruitment or classroom intervention.

Use of generative AI

OpenAI Codex assisted with code development, experiment orchestration, analysis, manuscript drafting and document preparation. Numerical results derive from executed experiments and archived outputs. The authors take responsibility for verifying the analysis and approving the manuscript.

References

[1] Christopher Bryant, Zheng Yuan, Muhammad Reza Qorib, Hannan Cao, Hwee Tou Ng, and Ted Briscoe. Grammatical error correction: A survey of the state of the art. *Computational Linguistics*, 49(3):643–701, September 2023. `https://doi.org/10.1162/coli_a_00478`. URL `https://aclanthology.org/2023.cl-3.4/`.

[2] Kostiantyn Omelianchuk, Vitaliy Atrasevych, Artem Chernodub, and Oleksandr Skurzhanskyi. GECToR – grammatical error correction: Tag, not rewrite. In Jill Burstein, Ekaterina Kochmar, Claudia Leacock, Nitin Madnani, Ildikó Pilán, Helen Yannakoudakis, and Torsten Zesch, editors, *Proceedings of the Fifteenth Workshop on Innovative Use of NLP for Building Educational Applications*, pages 163–170, Seattle, WA, USA → Online, July 2020. Association for Computational Linguistics. `https://doi.org/10.18653/v1/2020.bea-1.16`. URL `https://aclanthology.org/2020.bea-1.16/`.

[3] Xiuyu Wu, Jingsong Yu, Xu Sun, and Yunfang Wu. Position offset label prediction for grammatical error correction. In Nicoletta Calzolari, Chu-Ren Huang, Hansaem Kim, James Pustejovsky, Leo Wanner, Key-Sun Choi, Pum-Mo Ryu, Hsin-Hsi Chen, Lucia Donatelli, Heng Ji, Sadao Kurohashi, Patrizia Paggio, Nianwen Xue, Seokhwan Kim, Younggyun Hahm, Zhong He, Tony Kyungil Lee, Enrico Santus, Francis Bond, and Seung-Hoon Na, editors, *Proceedings of the 29th International Conference on Computational Linguistics*, pages 5409–5418, Gyeongju, Republic of Korea, October 2022. International Committee on Computational Linguistics. URL `https://aclanthology.org/2022.coling-1.480/`.

[4] Hiroki Homma and Mamoru Komachi. Non-autoregressive grammatical error correction toward a writing support system. In Erhong YANG, Endong XUN, Baolin ZHANG, and Gaoqi RAO, editors, *Proceedings of the 6th Workshop on Natural Language Processing Techniques for Educational Applications*, pages 1–10, Suzhou, China, December 2020. Association for Computational Linguistics. `https://doi.org/10.18653/v1/2020.nlptea-1.1`. URL `https://aclanthology.org/2020.nlptea-1.1/`.

[5] Mio Arai, Masahiro Kaneko, and Mamoru Komachi. Grammatical-error-aware incorrect example retrieval system for learners of Japanese as a second language. In Helen Yannakoudakis, Ekaterina Kochmar, Claudia Leacock, Nitin Madnani, Ildikó Pilán, and Torsten Zesch, editors, *Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications*, pages 296–305, Florence, Italy, August 2019. Association for Computational Linguistics. https://doi.org/10.18653/v1/W19-4431. URL https://aclanthology.org/W19-4431/.

[6] Aomi Koyama, Tomoshige Kiyuna, Kenji Kobayashi, Mio Arai, and Mamoru Komachi. Construction of an evaluation corpus for grammatical error correction for learners of Japanese as a second language. In Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis, editors, *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 204–211, Marseille, France, May 2020. European Language Resources Association. ISBN 979-10-95546-34-4. URL https://aclanthology.org/2020.lrec-1.26/.

[7] Daisuke Suzuki, Yujin Takahashi, Ikumi Yamashita, Taichi Aida, Tosho Hirasawa, Michitaka Nakatsuji, Masato Mita, and Mamoru Komachi. Construction of a quality estimation dataset for automatic evaluation of Japanese grammatical error correction. In Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Jan Odijk, and Stelios Piperidis, editors, *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 5565–5572, Marseille, France, June 2022. European Language Resources Association. URL https://aclanthology.org/2022.lrec-1.596/.

[8] Alex Kimn, Yiqun Hu, and Takako Aikawa. Rule-based synthetic data for japanese gec. Zenodo, 2020. URL https://zenodo.org/records/4276130.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. arXiv:2106.09685, 2021. URL https://arxiv.org/abs/2106.09685.

[10] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar, October 2014. Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1179. URL https://aclanthology.org/D14-1179/.

[11] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. https://doi.org/10.1162/neco.1991.3.1.79. URL https://www.cs.toronto.edu/~hinton/absps/jjnh91.pdf.

[12] Qwen Team. Qwen3-4B-Instruct-2507 model card, 2025. URL https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507. Commit cdbee75f17c01a7cc42f958dc650907174af0554. Accessed 28 September 2026.

[13] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. URL https://nlp.stanford.edu/IR-book/information-retrieval-book.html.

[14] Hideo Teramura. Gaikokujin gakushusha no nihongo goyoreishu [collection of japanese errors by foreign learners]. NINJAL digital repository, 1990. URL https://www2.ninjal.ac.jp/teramuradb/about.html. Database digitization 2011. Publisher text and web annotations accessed 28 September 2026.